

Orchestrator for Red Hat Developer Hub

Version 1.0.0, 2026-06-08

Home

Introduction

Course Title: Orchestrator for Red Hat Developer Hub

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Introduction
- Installation
- Automating workflow
- Debugging workflow

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience...
- Course or experience...
- Course or experience...

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Introduction

Introduction to Orchestrator

Orchestrator for Red Hat Developer Hub (RHDH) is a powerful tool designed to streamline development processes through serverless workflows. It acts as a bridge between complex automation tasks and developer self-service, providing the following capabilities:

- **Serverless Workflows:** Enables the execution of event-driven, automated processes within RHDH without managing underlying infrastructure.
- **Operational Efficiency:** Simplifies cloud migration and team onboarding by providing repeatable, standardized automation patterns.
- **Automation Framework:** Facilitates the creation, management, and monitoring of technical workflows directly integrated into the developer portal.
- **Scalability:** Offers a robust foundation for orchestrating complex CI/CD pipelines and software template bootstrapping.

Understanding Orchestrator features and benefits

Orchestrator for Red Hat Developer Hub

Understanding Orchestrator features and benefits

The Orchestrator plugin in Red Hat Developer Hub (RHDH) serves as the engine for automating complex, multi-step processes. By bridging the gap between manual human intervention and automated infrastructure tasks, Orchestrator provides a centralized framework to standardize development operations.

Core Features

Orchestrator is designed to enhance the RHDH ecosystem with the following capabilities:

- **Workflow Automation:** Design and execute complex logic that spans across multiple applications, services, and infrastructure components. This moves beyond simple scripts by providing a stateful, manageable execution environment.
- **Standardization:** Create uniform pathways for common technical tasks. By codifying processes such as developer onboarding or application migration, you ensure consistency and reduce the "configuration drift" often associated with manual setup.
- **Workflow Chaining:** Orchestrator supports the interconnection of multiple workflows. By adhering to the `WorkflowResult` schema, the output of one process can seamlessly trigger or inform the execution of the next.
- **Enterprise-Grade Scalability:** Built for the RHDH environment, Orchestrator supports the collaboration needs of large organizations, allowing teams to scale their automation efforts without sacrificing visibility.

Key Benefits

Implementing Orchestrator within your RHDH instance provides measurable improvements to developer experience and operational efficiency:

Benefit	Description
Reduced Manual Effort	Automating multi-step processes minimizes repetitive human tasks, allowing developers to focus on feature delivery rather than infrastructure plumbing.
Enhanced Visibility	Through the Orchestrator UI, teams can monitor the status of active workflows, review execution logs, and diagnose bottlenecks in real-time.
Improved Reliability	By standardizing migrations and onboarding, you eliminate the human error inherent in manual configuration, leading to more predictable and stable deployments.
Unified Error Handling	By structuring outputs to include error information, Orchestrator enables the UI and subsequent workflows to react intelligently to failures, improving the overall resiliency of your automated pipelines.

Best Practices for UI Integration

To leverage the full power of the Orchestrator UI, adhere to these architectural standards:

1. **Strict Schema Adherence:** Ensure that all workflow results conform to the `WorkflowResult` schema. This allows the RHDH dashboard to render your workflow data in a clear, actionable format.
2. **Explicit Error Reporting:** Always include error diagnostic information within the workflow output. This practice is essential for effective debugging and ensures that downstream workflows (if chained) can make informed decisions when a preceding step fails.

By utilizing these features, teams can transform RHDH from a simple software catalog into a powerful orchestration platform that drives consistent and scalable engineering practices across the entire organization.

Overview of serverless workflows in RHDH

Overview of serverless workflows in RHDH

Red Hat Developer Hub (RHDH) leverages the Orchestrator plugin to provide robust automation capabilities. At its core, this automation is powered by **Serverless Workflows**, which allow developers to define, execute, and monitor complex business logic as code.

What are Serverless Workflows?

Serverless Workflows in RHDH are based on the CNCF Serverless Workflow specification. They provide a declarative way to define sequences of tasks—such as service calls, wait states, and

conditional logic—without managing the underlying infrastructure.

In the RHDH ecosystem, these workflows act as the "glue" that connects various internal and external developer services.

Core Components and Architecture

The RHDH Orchestrator relies on the **OpenShift Serverless Logic operator**, which provides the runtime environment necessary to execute these workflows. The architecture integrates several critical components:

- **OpenShift Serverless (Knative):** Serves as the backbone for event-driven communication. It utilizes **Knative Eventing** to interface seamlessly with the Data Index service, ensuring that workflow state and historical execution data are persisted and queryable.
- **Knative Functions:** These are utilized to inject complex, custom logic into workflows that cannot be achieved via standard workflow states alone. If a workflow requires a specific calculation or a unique integration not supported by default, a Knative function can be invoked.
- **Data Index Service:** Acts as the persistence layer, indexing the state of workflows and allowing RHDH to provide a unified view of execution history and runtime status.

Key Capabilities

Resilience and Fault Tolerance

A primary design philosophy of Serverless Workflow is **resilience**. The engine is built to handle transient failures through: * **Retries:** Automatic configuration to attempt failed tasks again after a specified duration. * **Error Handling:** Custom compensation logic that triggers when a workflow branch fails, ensuring that the system can roll back or clean up resources to maintain a consistent state.

Event-Driven Integration

By utilizing Knative, workflows are inherently event-driven. They can be triggered by external events (e.g., a webhook from a Git repository or a completed CI/CD job) and can emit events upon completion. This decoupling allows RHDH workflows to participate in broader, enterprise-wide event meshes.

Scalability

Because these workflows run on top of OpenShift Serverless, they benefit from "scale-to-zero" capabilities. Resources are consumed only when a workflow is actively executing a task, making the Orchestrator an efficient solution for both high-traffic automation and intermittent background tasks.

Summary of Benefits for RHDH

By adopting Serverless Workflows, RHDH users gain: * **Standardization:** Using a common, declarative specification for automation across the organization. * **Operational Simplicity:** Reducing the need to maintain persistent cron jobs or long-running automation servers. * **Visibility:** Deep integration with RHDH dashboards, allowing teams to track the lifecycle of their

onboarding, migration, or provisioning processes in real-time.



For advanced scenarios, refer to the **OpenShift Serverless** documentation on building and deploying serverless workflows to learn how to create custom service extensions that your RHDH Orchestrator can trigger.

Use cases for cloud migration and onboarding

Use cases for cloud migration and onboarding

Orchestrator in Red Hat Developer Hub (RHDH) serves as the engine for automating complex, multi-step processes. By leveraging serverless workflows, organizations can standardize and accelerate high-friction operations like cloud migration and developer onboarding.

Cloud Migration Use Cases

Cloud migration often involves a repetitive, manual sequence of infrastructure provisioning, code refactoring, and security validation. Orchestrator simplifies this by turning these sequences into repeatable, auditable workflows.

- **Infrastructure Provisioning:** Orchestrator can trigger Infrastructure-as-Code (IaC) pipelines (e.g., Terraform or Ansible) to create the necessary cloud environment, ensuring that the target environment meets the organization's compliance standards before the application code is deployed.
- **Database Migration Coordination:** Use workflows to orchestrate the "lift-and-shift" or "refactor" process. A workflow can coordinate data integrity checks between the on-premise source and the cloud-native destination, ensuring minimal downtime.
- **Automated Validation:** Once components are moved, workflows can automatically execute smoke tests and performance benchmarks, reporting the results back to the RHDH dashboard for stakeholders to approve the migration status.

Developer Onboarding Use Cases

The onboarding process is frequently hampered by context switching and manual configuration. Orchestrator reduces the "time-to-first-commit" by automating the setup of a developer's workspace and access rights.

- **Standardized Project Bootstrapping:** Integrate Orchestrator with RHDH Software Templates to ensure that every new project starts with the correct folder structure, documentation, CI/CD configuration, and security headers.
- **Access Control and Identity Management:** Automate the request-approval cycle for repository access, cloud project permissions, and service account credentials. When a new developer joins a team, a single workflow can provision these resources across multiple integrated systems.
- **Toolchain Integration:** Automatically register new repositories in monitoring tools (such as Grafana or Loki) and alert systems as part of the onboarding workflow, ensuring that the developer is "day-one ready."

Hands-on Activity: Defining a Migration/Onboarding Trigger

This activity demonstrates how to initiate a workflow that mimics an automated project onboarding request.



Ensure you have the **Orchestrator** plugin enabled and the necessary **Software Templates** charts installed in your RHDH environment.

1. **Create a Workflow Definition** Create a new workflow file (e.g., `onboarding-workflow.yaml`) that defines the steps for a new project:

```
name: onboarding-workflow
steps:
  - name: create-repo
    action: github:create-repo
  - name: scaffold-template
    action: scaffolder:run-template
    input:
      templateUrl: 'url-to-your-standard-template'
  - name: send-notification
    action: slack:send-message
    input:
      message: 'New project environment is ready for development.'
```

2. **Deploy the Workflow** Upload the workflow definition to your Orchestrator instance. **Navigate to your RHDH dashboard → Orchestrator → Workflows → Create New.**
3. **Execute the Workflow** Trigger the workflow by providing the required inputs (e.g., `project-name`, `team-owner`, `cloud-region`). Once submitted, monitor the progress in the **Workflow Instances** view to see each step—from repository creation to final notification—executing in real-time.
4. **Verify Results** Check the linked GitHub repository and verify that the initial CI/CD pipelines have been bootstrapped according to your organization's standards.



For complex cloud migrations, decompose large tasks into smaller, modular workflows. This makes debugging individual steps significantly easier if a network timeout or credential error occurs during the migration process.

Installation

Installation

You can deploy Red Hat Developer Hub (RHDH) with Orchestrator by utilizing the following methods:

- **RHDH Operator:** Install the Orchestrator component directly through the Red Hat Developer Hub Operator, which may require manual approval of install plans via `oc patch` commands.
- **Helm Chart:** Utilize the RHDH Helm chart to manage the deployment of the Orchestrator.
- **Network Configurations:** The installation supports deployment across various OpenShift network environments, including:
 - Fully disconnected environments.
 - Partially disconnected environments.
- **Helper Script:** Components can also be installed using the dedicated RHDH helper script as outlined in the technical appendix.

Deploying RHDH with Orchestrator via Helm chart

Deploying RHDH with Orchestrator via Helm chart

The Red Hat Developer Hub (RHDH) Orchestrator plugin extends the capabilities of RHDH by providing serverless workflow automation. Deploying the Orchestrator requires careful configuration of the Helm chart, particularly when managing container image sources for air-gapped or partially disconnected environments.

Prerequisites

- An active OpenShift cluster.
- Helm CLI installed and configured.
- Access to your OpenShift cluster as a `cluster-admin`.
- A custom registry (if operating in a disconnected environment) to host Orchestrator plugin OCI images.

Installing the Orchestrator Templates

Before deploying the core Orchestrator service, you must deploy the software templates that define your workflows.

1. Add the repository:

```
helm repo add redhat-developer https://redhat-developer.github.io/rhdh-chart
helm repo update
```

2. Install the software templates chart:

```
helm install my-orchestrator-templates redhat-developer/orchestrator-software-templates --version 0.2.0
```

3. Configure environment-specific values:

Retrieve your RHDH route and map it to your template values:

```
RHDH_ROUTE="https://$(oc get route -n <your-rhdh-namespace> -o jsonpath='{.items[0].spec.host}')"
cp charts/orchestrator-software-templates/orchestrator-templates-values.yaml.template orchestrator-templates-values.yaml
sed -i "s|__RHDH_BASE_URL__|$RHDH_ROUTE|g" orchestrator-templates-values.yaml
```

Deploying RHDH with Orchestrator

When deploying the main RHDH instance, you must explicitly enable the Orchestrator plugin within your `values.yaml` file.

Configuring Custom Registries

By default, RHDH pulls Orchestrator plugins from the official Red Hat OCI registry. For disconnected or internal-registry-gated environments, you must override these references.

1. **Modify your `values.yaml`:** Add the `orchestrator` section to your RHDH helm configuration. Replace the simplified plugin references with the full OCI URLs pointing to your internal registry.

```
orchestrator:
  enabled: true
  plugins:
    - package: <target-registry-url:port>/<namespace>/<plugin-name>:<version>
      pluginConfig:
        # Plugin-specific configuration goes here
```

2. **Apply the Infrastructure Chart:** Ensure the `redhat-developer-hub-orchestrator-infra` Helm chart is applied to your cluster to prepare the necessary CRDs and underlying resources.
3. **Apply the RHDH 1.9 Helm Chart:** Install or upgrade your RHDH instance, ensuring the Orchestrator toggle is enabled:

```
helm upgrade --install rhdh redhat-developer/redhat-developer-hub \
  --version 1.9.3 \
  --set orchestrator.enabled=true \
  -f values.yaml
```

Troubleshooting Common Deployment Issues

- **Image Pull Back-offs:** If pods fail to start, verify that the `orchestrator.plugins` list uses the exact FQDN of your local registry. Ensure that your cluster has the necessary pull secrets to access that registry.
- **Missing Templates:** If Orchestrator workflows do not appear in the RHDH UI, verify that the `orchestrator-software-templates` chart was successfully installed in the same namespace as your RHDH deployment and that the `RHDH_BASE_URL` matches the actual route.
- **Version Mismatches:** Ensure your Orchestrator plugin versions are compatible with RHDH 1.9. Refer to the specific plugin documentation for compatibility matrices.

Configuring for fully disconnected OpenShift environments

Configuring Orchestrator for Fully Disconnected OpenShift Environments

In a fully disconnected (air-gapped) environment, Red Hat Developer Hub (RHDH) and the Orchestrator plugin cannot reach the public internet to pull container images or operator dependencies. To deploy successfully, you must mirror all necessary artifacts to a local private registry and configure the OpenShift cluster to trust and pull from this location.

Prerequisites

- You have a fully disconnected OpenShift Container Platform (OCP) cluster.
- You have a local container registry accessible by the cluster.
- You have `oc-mirror` installed, matching your OCP version.
- You have administrator privileges to the cluster.

Procedure: Preparing the Disconnected Environment

The following steps outline the requirements for enabling Orchestrator in a restricted network.

1. Mirroring Necessary Components

Before installing the RHDH Operator or its dependencies, use the `oc-mirror` tool to populate your local registry with the required images. Ensure the following operators are included in your `imageSetConfiguration.yaml`:

- Red Hat OpenShift Serverless Operator
- Red Hat OpenShift Serverless Logic Operator
- Red Hat Developer Hub Operator

2. Installing Serverless Components

Orchestrator relies on serverless workflows. You must prepare the underlying infrastructure:

1. Install the OpenShift Serverless Operator via OperatorHub (using your local registry mirror).
2. Create an instance of the **KnativeServing** and **KnativeEventing** custom resources to enable the serverless capabilities required by the Orchestrator engine.

3. Redirecting Image Pulls

To ensure the cluster pulls from your private registry instead of the public Red Hat registry, apply the generated cluster resources:

```
# Navigate to the directory where oc-mirror generated the manifests
cd <workspace-folder>/working-dir/cluster-resources/

# Apply the ImageContentSourcePolicy or ImageDigestMirrorSet
oc apply -f .
```

4. Installing Orchestrator Dependencies

Install the required Operators via the OperatorHub interface, ensuring they point to your internal catalog source:

1. Install the **OpenShift Serverless Operator**.
2. Install the **OpenShift Serverless Logic Operator**.
3. Install the **Red Hat Developer Hub Operator**.

5. Configuring the Backstage Custom Resource (CR)

Once the operators are running, configure the **Backstage** CR to include the Orchestrator plugin. When operating in a disconnected mode, ensure that any external plugin references are mapped to your local registry.

```
apiVersion: rhdh.redhat.com/v1alpha1
kind: Backstage
metadata:
  name: developer-hub
spec:
  application:
    extraEnvs:
      envs:
        - name: ORCHESTRATOR_ENABLED
          value: "true"
    # Ensure any custom images for plugins are pointed at your local registry
    image: <target-registry-url:port>/rhdh/rhdh-hub@sha256:<digest>
  # Configure plugin-specific settings
  deployment:
    patch:
      spec:
        template:
          spec:
```

```
containers:
  - name: backstage-backend
    env:
      - name: KOGITO_SERVICE_URL
        value: "http://orchestrator-service"
```



Replace `<target-registry-url:port>` with your actual internal registry address (e.g., `registry.internal.example.com:5000`). Failure to do so will result in `ImagePullBackOff` errors during the Orchestrator pod initialization.

Troubleshooting Disconnected Configuration

If the Orchestrator fails to start:

- **Check ImageStream Mappings:** Ensure the `ImageContentSourcePolicy` is active. Verify with `oc get imagesourcecontentpolicy`.
- **Verify Registry Connectivity:** Run a test pod in the namespace to ensure the nodes can pull an image from the local registry using `oc run test-pod --image=<target-registry-url:port>/test -image:latest`.
- **Review Logs:** Use `oc logs -n <rhhd-namespace> <orchestrator-pod-name>` to identify if the service is failing due to authentication issues with the internal registry.

Configuring for partially disconnected OpenShift environments

Configuring for partially disconnected OpenShift environments

In partially disconnected environments, your OpenShift cluster has limited or restricted access to the public internet, requiring all container images and dependencies to be pulled from an internal, secure mirror registry. To successfully deploy Red Hat Developer Hub (RHDH) with the Orchestrator plugin, you must ensure that the environment is prepared to handle image redirections and that all necessary operators are available locally.

Prerequisites

Before beginning the installation, ensure the following requirements are met:

- Your disconnected environment is configured with a local, private container registry.
- You possess the necessary permissions to push OCI images to your internal registry.
- The `oc-mirror` tool is installed, and its version matches your OpenShift Container Platform (OCP) cluster version.
- You have identified your target registry URL and port (e.g., `registry.example.com:5000`).

Procedure: Configuring the Environment

Follow these steps to prepare your environment and deploy the Orchestrator plugin within a partially disconnected OpenShift cluster.

1. Mirroring Resources

Use the `oc-mirror` tool to sync the required images to your local registry.

1. Create your `ImageSetConfiguration` file to include the RHDH and Orchestrator-related images.
2. Execute the mirror command:

```
oc-mirror --config mirror-config.yaml docker://<target-registry-url:port>
```

2. Redirecting Image Pulls

Once the images are mirrored, you must instruct the cluster to pull images from your local registry instead of the default external sources.

1. Navigate to the working directory generated by the `oc-mirror` tool:

```
cd <workspace folder>/working-dir/cluster-resources/
```

2. Apply the generated `ImageContentSourcePolicy` and other cluster-wide resources:

```
oc apply -f .
```

3. Installing Operators

With the registry configured, install the foundational operators required for Orchestrator functionality via the OpenShift OperatorHub.

1. **OpenShift Serverless Operator:** Install this first to provide the underlying serverless capabilities.
2. **OpenShift Serverless Logic Operator:** Install this to enable the workflow automation engine required by the Orchestrator plugin.

4. Deploying the Backstage Custom Resource (CR)

After the operators are running, you must configure the Backstage instance to recognize the Orchestrator plugin.

1. Create a `Backstage` Custom Resource.
2. Configure the CR to include the Orchestrator plugin dependencies. Refer to the specific plugin documentation for the exact YAML configuration required for your version of RHDH.



If your organization requires persistent data storage that is not managed by the

default RHDH setup, you may optionally deploy a custom PostgreSQL database instance within your cluster at this stage. Ensure that the database service is reachable by the Backstage pod.

5. Finalizing Installation

Apply the final resource configuration:

```
oc apply -f backstage-cr.yaml
```

Once applied, monitor the deployment using the OpenShift web console or CLI:

```
oc get pods -n <your-rhdh-namespace>
```

Ensure the **backstage** pod and any associated **orchestrator** pods transition to the **Running** state. If the pods remain in **ImagePullBackOff**, verify that your **ImageContentSourcePolicy** was successfully applied and that your local registry contains the required images.

Automating workflow

Automating workflow

The Orchestrator plugin streamlines the software development lifecycle (SDLC) by automating the creation and management of serverless workflows. Key automation capabilities include:

- **Project Bootstrapping:** Use software templates to initialize complete projects, including pre-configured Git repositories and deployment configurations.
- **CI/CD Integration:** Automatically set up continuous integration and continuous deployment pipelines for efficient workflow delivery.
- **Workflow Lifecycle:** Manage the end-to-end process of creating, executing, and maintaining serverless workflows.
- **Simplified Design:** Utilize the Serverless Workflow DSL, which employs imperative verbs like `Call`, `Run`, and `Switch` to ensure workflows remain clear and easy to maintain.

Creating a workflow

Creating a workflow

To automate processes within Red Hat Developer Hub (RHDH), you must first define your workflow using the Serverless Workflow Domain Specific Language (DSL). The DSL is designed for clarity, utilizing imperative verbs such as `Call`, `Emit`, `Fork`, `Switch`, and `Wait` to ensure your automation logic is readable and maintainable.

Prerequisites

Before creating your first workflow, ensure you have the following tools installed: * **kn-workflow CLI:** The primary command-line tool for scaffolding and managing workflow projects. * **Quarkus:** The underlying framework used for building and running your workflow services. * **Container Engine:** (e.g., Podman or Docker) for building and testing images locally.

Workflow Design Principles

When designing your workflow, keep the structure modular. The Serverless Workflow DSL encourages a declarative approach, allowing you to define the state transitions and execution logic without deep coupling to the underlying infrastructure. By following the standard Quarkus project structure, you ensure that your workflow is ready for integration with RHDH templates and CI/CD pipelines.

Procedure: Creating a New Workflow Project

Follow these steps to initialize a new workflow project using the `kn-workflow` CLI.

1. **Scaffold the project:** Use the `kn-workflow` CLI to generate a new project. This command creates a standard Quarkus-based directory structure required for RHDH workflows.


```
kn-workflow quarkus create --name 00_new_project
```

1. **Develop the workflow:** Navigate to the project folder. You will find the workflow definition files (typically in JSON or YAML format) where you can define your process logic.



Ensure your workflow definition utilizes the core imperative verbs to maintain linguistic fluency and clarity for other developers auditing your code.

2. **Validate and run locally:** Before deploying to RHDH, test the logic in your local environment. The `run` command compiles your project and starts the workflow engine in development mode.

```
kn-workflow quarkus run
```



When executing `kn-workflow quarkus run`, the CLI pulls the necessary development image to simulate the RHDH runtime environment: `registry.redhat.io/openshift-serverless-1/logic-swf-devmode-rhel9:1.37.2`

Hands-on Activity: Configuring the Workflow Schema

In this activity, you will verify the project structure and prepare the workflow for your first execution.

1. **Navigate to your project directory:**

```
cd 00_new_project
```

2. **Add schema definitions:** Open the `workflow.sw.json` file in your editor. Define the `states` and `transitions`. For example, ensure your `functions` section references the necessary services that your workflow will `Call`.
3. **Verify local execution:** Once you have defined your state logic, verify that the application compiles successfully:

```
./mvnw clean install  
kn-workflow quarkus run
```

1. **Review:** Once the command is running, the workflow is active and listening for events or incoming triggers. You can now proceed to define the specific software templates or CI/CD pipelines that will trigger this workflow from within Red Hat Developer Hub.

Running a workflow

Running a Workflow in Red Hat Developer Hub

Once a Serverless Workflow has been defined and its project structure initialized, the next phase is execution. Running a workflow allows you to validate the orchestration logic, test task transitions, and ensure that the integration points—such as the Job Service—are interacting correctly with your business logic.

Prerequisites

Before attempting to execute a workflow, ensure the following requirements are met:

- * **kn-workflow CLI:** Installed and accessible in your local environment.
- * **Workflow Project:** A valid workflow definition (e.g., `workflow.sw.json` or `.yaml`) created via the CLI.
- * **Connectivity:** Ensure your local environment has the necessary permissions to communicate with the target Orchestrator instance if testing against an OpenShift cluster.

Execution via kn-workflow CLI

The `kn-workflow` CLI is the primary interface for managing the lifecycle of your workflow. To execute and test your workflow locally, you use the build and run capabilities provided by the tool.



When executing a workflow, always ensure your current working directory is the root of the project to allow the CLI to resolve dependencies and configuration files correctly.

Step-by-Step: Running the Basic Workflow

Follow these steps to build and execute your workflow project:

1. **Navigate to the Project Root:** Open your terminal and move to the directory where your workflow project is initialized.
2. **Build the Workflow:** Before running, ensure the project manifests are compiled correctly. Use the `-w` flag to point to the workflow directory and the `-m` flag to specify the output directory for your manifests: `[source,bash] ---- kn-workflow build -w ./src/main/resources -m ./target ----`
3. **Execute the Workflow:** To run the workflow, use the `run` command. This initiates the workflow engine locally, allowing you to trigger the workflow execution: `[source,bash] ---- kn-workflow run ----`

Orchestration and Execution Logic

When you trigger a workflow, the Orchestrator performs several internal tasks:

- **Instantiation:** The engine reads the workflow definition and creates an instance ID for tracking.
- **Job Service Integration:** If the workflow includes scheduled tasks, the Orchestrator registers these with the **Job Service**. The Job Service monitors the timers and triggers the workflow transitions at the specified intervals.
- **Execution Control:** The `orchestrator.workflow.use` capability handles the runtime state transitions. It manages the execution lifecycle, allowing the engine to transition between states,

update the context data, and—if necessary—abort the workflow if a failure threshold is reached.

Validating Execution

After triggering the workflow, you can verify its status by observing the output logs generated in your terminal.

- **Success:** You should see a confirmation message indicating that the workflow instance was created and the first state transition was successful.
- **Failure:** If the workflow fails to start, the CLI will output specific error codes related to the workflow definition or the connection to the Orchestrator service.



If you encounter unexpected behavior during execution, use the verbose flag (`--verbose` or `-v`) with the `kn-workflow` command to see detailed stack traces and debugging information during the runtime process.

Troubleshooting Common Execution Issues

If the workflow fails to run: 1. **Verify JSON/YAML Syntax:** Ensure your workflow definition follows the Serverless Workflow specification strictly. 2. **Check Port Availability:** If the local runtime requires a specific port, ensure no other service is binding to it. 3. **Check Orchestrator Connectivity:** If running against a remote RHDH instance, verify that your `oc` login session is active and has the required `edit` permissions on the namespace.

Managing a workflow

Managing a Workflow in RHDH

Managing a workflow within Red Hat Developer Hub (RHDH) involves monitoring execution, controlling runtime flow, and ensuring that tasks complete according to the defined Serverless Workflow DSL. The `backstage-plugin-orchestrator` serves as the primary interface for these administrative tasks.

Overview of Workflow Management

Once a workflow is deployed, lifecycle management is handled through the Orchestrator frontend plugin. You can perform the following actions to manage your automated processes:

- **Execution Monitoring:** Track the state of active and completed workflow instances.
- **Runtime Control:** Observe data flow between tasks, including `Do`, `Listen`, `Raise`, `Run`, `Try`, and `Wait` operations.
- **Status Tracking:** Review the execution status to identify whether a workflow is pending, in progress, completed, or failed.
- **Log Inspection:** Access runtime logs to verify the output of individual tasks.

Hands-on Activity: Managing Workflow Instances

This activity guides you through the process of monitoring and managing a running workflow instance using the RHDH Orchestrator interface.

Prerequisites

- A running instance of Red Hat Developer Hub.
- The `backstage-plugin-orchestrator` installed and configured.
- At least one workflow successfully deployed and triggered.

Step 1: Accessing the Orchestrator Dashboard

1. Log in to your RHDH portal.
2. In the left-hand navigation menu, click on **Orchestrator**.
3. Navigate to the **Workflows** tab to see a list of all defined workflows.

Step 2: Monitoring an Active Workflow

1. Select the specific workflow you wish to manage.
2. Click on the **Instances** tab to view all historical and current executions of that workflow.
3. Click on a specific **Instance ID** to open the details view.
4. Observe the visual representation of the workflow. The diagram highlights the current task being executed by the runtime.

Step 3: Inspecting Task Data Flow

1. Within the instance view, click on a completed or active task node.
2. Review the **Task Details** panel.
 - **Tip:** If the task type is `Run` or `Do`, verify the input/output data parameters to ensure the payload was processed correctly.
3. If the workflow is stalled, check the `Wait` or `Listen` task status to confirm if the workflow is waiting for an external event or timeout.

Step 4: Troubleshooting Workflow Status

If a workflow displays a `Failed` status:

1. Navigate to the **Logs** tab within the specific instance view.
2. Analyze the error message. Common issues include:
 - * **Data schema mismatch:** The input provided does not match the required task format.
 - * **Timeout:** The workflow exceeded the allowed time defined in a `Wait` task.
 - * **Connectivity:** The RHDH service could not reach the endpoint defined in a `Run` task.



To maintain efficiency, always verify that your task definitions adhere to the Serverless Workflow DSL principles. Keeping tasks granular makes managing and troubleshooting significantly easier when scaling workflows.

Next Steps

After mastering the management of workflow instances, you can progress to automating your deployment lifecycle by integrating these workflows with your existing CI/CD pipelines.

Bootstrapping projects with software templates

Bootstrapping projects with software templates

In Red Hat Developer Hub (RHDH), Orchestrator leverages software templates to standardize the creation of serverless workflows. By using templates, developers can bootstrap a project structure that includes pre-configured workflow definitions, Kustomize configurations, and integrated CI/CD pipelines, ensuring consistency across the development lifecycle.

Prerequisites

Before bootstrapping a project, ensure your environment is correctly configured:

- The `orchestrator-software-templates` Helm chart must be installed in your RHDH instance. This chart populates the RHDH Catalog with the necessary workflow templates.
- The `orchestratorauth-secret` must exist within the target namespace to facilitate secure communication between the workflow engine and the RHDH backend.



If your software templates are not visible in the Catalog, verify that the `orchestrator-software-templates` chart deployment was successful and that the `orchestratorauth-secret` is correctly mounted in the namespace where your RHDH instance is running.

Hands-on Activity: Bootstrapping a New Workflow Project

This activity guides you through using an existing software template to create a new workflow project.

Step 1: Access the Catalog

1. Navigate to your RHDH dashboard.
2. In the left-hand sidebar, click on **Catalog**.
3. Use the filter to select **Templates** to view available bootstrapping options.

Step 2: Select the Orchestrator Template

1. Look for the template labeled "**Orchestrator Workflow Project**" (or the equivalent provided by your organization's `orchestrator-software-templates` chart).
2. Click **Choose** to initiate the scaffolding process.

Step 3: Configure Project Details

Fill in the required parameters in the form provided: * **Project Name:** Provide a unique name for

your workflow project. * **Owner:** Assign the team or user responsible for the workflow. * **Repository:** Select the Git provider and organization where the project code should be hosted. * **Workflow Name:** Define the initial name for your Serverless Workflow definition.

Step 4: Review and Generate

1. Review the generated configuration summary.
2. Click **Create** to trigger the scaffold process.
3. RHDH will automatically push the generated files—including your `workflow.sw.yaml`, Kustomize overlays, and CI/CD pipeline definitions—to your chosen Git repository.

Understanding the Bootstrapped Structure

Once the template has finished generating the project, the repository will typically contain the following structure:

- `/workflow`: Contains your Serverless Workflow definition files.
- `/k8s`: Contains Kustomize configurations for deploying the workflow to OpenShift.
- `/.tekton`: Contains the pre-configured CI/CD pipeline definitions to automate testing and deployment.

By bootstrapping with these templates, you ensure that every new workflow project adheres to your organization's operational standards, including security configurations and logging requirements.

Troubleshooting Template Issues

If the project creation fails or the template does not trigger:

- **Verify Logs:** Check the RHDH backend logs for errors related to the template processor.
- **Check Permissions:** Ensure the service account used by RHDH has sufficient permissions to push code to your Git provider.
- **Validate Secrets:** Confirm that the `orchestratorauth-secret` is correctly configured, as this is required for the template engine to authenticate with the Orchestrator backend during the scaffolding process.

Setting up CI/CD pipelines for workflows

Setting up CI/CD pipelines for workflows

Automating the deployment of workflows in Red Hat Developer Hub (RHDH) requires a robust CI/CD pipeline that integrates your source code management with GitOps practices. This section outlines how to orchestrate your workflow lifecycle using CI tools and Argo CD.

Overview of the CI/CD Pipeline

The workflow lifecycle consists of two primary phases: * **Continuous Integration (CI):** Validates

your workflow code, runs tests, and builds the necessary artifacts. * **Continuous Deployment (CD):** Uses Argo CD (or Red Hat OpenShift GitOps) to synchronize your workflow resources into the OpenShift cluster.

Procedure: Setting up CI and CD

1. Implementing Continuous Integration (CI)

To ensure your workflow is deployment-ready, follow these steps to verify your CI pipeline:

1. In the RHDH Catalog, navigate to your source code repository component (e.g., **onboardings**).
2. Click the **CI** tab to view the pipeline run status.
3. Ensure the pipeline run displays **Succeeded**.
4. If the pipeline status does not reflect in the RHDH console, verify the status directly within your Git provider (GitHub, GitLab, or your configured CI tool).
5. If the pipeline fails, click the run name to inspect the logs for build errors or configuration mismatches.



You can extract the steps from the provided build script in your repository and implement them in your preferred CI/CD tool, such as Tekton, GitHub Actions, or Jenkins.

2. Configuring Continuous Deployment (CD)

Once the CI pipeline succeeds, the deployment to your cluster is handled via GitOps.

1. **Label the Target Namespace:** Argo CD requires specific labeling to recognize the target namespace. Run the following command in your terminal:

```
oc label ns <target_namespace> rhdh.redhat.com/argocd-namespace=true
```

2. **Verify Permissions:** Ensure the Argo CD **ServiceAccount** possesses the necessary RBAC permissions to manage resources within the target **rhdh** namespace.
3. **Monitor Deployment in RHDH:**
 - Open the **GitOps Resources Repository** component (e.g., **onboarding-gitops**) in the RHDH Catalog.
 - Click the **CD** tab.
 - Confirm that the Kubernetes resources are listed as **Synced** and **Healthy**. This status indicates that Argo CD has successfully deployed the workflow to your cluster.

Troubleshooting Common CD Issues

If the CI pipeline succeeds but your workflow is missing from the **CD** tab:

- **Namespace Configuration:** Re-verify that the target namespace has been correctly labeled for Argo CD as shown in the step above.

- **RBAC Constraints:** Verify that your Argo CD **ServiceAccount** has sufficient cluster permissions to create the workflow resources in the destination namespace.
- **Log Inspection:** Check the Argo CD controller logs if the sync status remains in a **Progressing** or **Degraded** state for an extended period.

Debugging workflow

Debugging Orchestrator workflows

Debugging workflows in Red Hat Developer Hub involves identifying and resolving issues related to deployment, connectivity, and configuration. Effective debugging practices include:

- **Log Analysis:** Reviewing backend logs and configurations to identify execution failures.
- **Log Integration:** Integrating external log providers, such as Loki, to gain deeper insights into workflow behavior and monitoring.
- **Troubleshooting:** Addressing common deployment challenges, including plugin visibility, pipeline execution errors, and resource synchronization issues.

Reviewing logs and configs

Reviewing Logs and Configurations for Orchestrator Workflows

In Red Hat Developer Hub (RHDH), Orchestrator serves as the engine for serverless workflows. When a workflow fails or exhibits unexpected behavior, the primary path to resolution is a systematic review of the backend logs and the associated workflow configurations.

Understanding Orchestrator Logging

Orchestrator relies on the underlying RHDH backend infrastructure to capture execution traces. Logs are generated at the workflow execution level, capturing state transitions, task failures, and integration errors.

To effectively debug, you must distinguish between two primary sources of information: 1. **Workflow Execution Logs:** Generated by the workflow engine, providing insight into which specific step (task) failed within your workflow definition. 2. **Backend Infrastructure Logs:** Captured by the OpenShift pod running the RHDH backend, useful for identifying connectivity issues, authentication failures, or resource constraints.

Reviewing Workflow Configurations

Before diving into logs, verify that your workflow configuration is syntactically and logically correct. Orchestrator uses JSON/YAML-based workflow definitions.

- **Check Schema Validation:** Ensure your workflow definition complies with the expected schema. A common source of errors is incorrect mapping of input/output data between task nodes.
- **Verify Endpoint Configurations:** Ensure that any external services (e.g., CI/CD triggers, database connectors) defined in your configuration are reachable from the cluster.
- **Environment Variables:** Check the `app-config.yaml` or relevant Kubernetes Secrets/ConfigMaps to ensure that environment-specific variables used by the workflow (like API tokens or service URLs) are correctly injected.

Hands-On Activity: Analyzing Workflow Execution Logs

In this activity, you will access the logs for a running or failed Orchestrator workflow via the OpenShift CLI.

Prerequisites

- Access to an OpenShift cluster with RHDH installed.
- **oc** CLI tool configured with appropriate permissions.
- A workflow currently deployed in the **rhdh** namespace.

Steps

1. **Identify the RHDH backend pod:** List the pods in your namespace to find the Orchestrator backend pod: [source,bash] `oc get pods -n rhdh`
2. **Access real-time logs:** Stream the logs from the backend container to observe the workflow execution in real-time: [source,bash] `oc logs -f deployment/rhdh-backstage -n rhdh`
3. **Filter logs for specific Workflow IDs:** To reduce noise, pipe the log output to **grep** to find entries related to your specific workflow execution ID: [source,bash] `oc logs -f deployment/rhdh-backstage -n rhdh | grep "<WORKFLOW_INSTANCE_ID>"`
4. **Reviewing Configuration files in the UI:**
 - Navigate to the **Orchestrator** tab in the Red Hat Developer Hub interface.
 - Select the failed workflow instance.
 - Examine the **Execution Details** pane. If a step failed, check the task input/output payload displayed in the UI to see if the JSON data matches the expected schema.

Troubleshooting Tips

- **Task Timeouts:** If you see "Task execution timed out" in the logs, check the workflow node configuration for **timeout** parameters. You may need to increase the threshold for long-running processes.
- **Permission Denied:** If the logs indicate 401/403 errors, verify the service account tokens provided to the Orchestrator backend are still valid and possess the necessary RBAC roles to interact with the target resource.
- **Loki Integration:** For persistent debugging, ensure your RHDH instance is configured to forward logs to an external aggregator like Loki, allowing you to search through historical execution logs rather than relying solely on pod restart retention.

Troubleshooting the common issues

Troubleshooting Common Workflow Issues in RHDH

When managing serverless workflows within Red Hat Developer Hub (RHDH), encountering deployment or execution failures is part of the operational lifecycle. Effectively troubleshooting

these issues requires a systematic approach, ranging from visibility checks to deep-dive diagnostics.

Identifying Common Deployment Failures

Before diving into complex logs, verify the state of your workflow deployments. Common issues often stem from misconfigurations in the plugin layer or resource synchronization.

Troubleshooting Checklist

- **Plugin Visibility:** Ensure that the Orchestrator plugin is correctly registered and visible within the RHDH interface. If the UI does not show the Orchestrator tab, check the `app-config.yaml` for missing proxy or plugin definitions.
- **Pipeline Execution:** If a workflow triggers but stalls, verify that the underlying execution engine has the necessary RBAC permissions to interact with the target OpenShift namespaces.
- **Resource Synchronization:** Ensure your software templates are correctly referenced. A common failure occurs when the workflow definition points to a catalog entity that has been deleted or moved.

Advanced Diagnostics: Java Flight Recorder

For complex, deep-seated issues that are not immediately obvious via standard logs, you can utilize the **Java Flight Recorder (JFR)**.



JFR is particularly useful for identifying performance bottlenecks or memory-related stalls within the Java-based Orchestrator backend.

JFR operates by recording data over a specific time interval. Unlike standard thread dumps, JFR provides a comprehensive diagnostic snapshot:

1. **Configure Interval:** Define the capture window in your JFR configuration file based on when the workflow failure typically occurs.
2. **Enable Recording:** Use the JFR diagnostic command to start recording for the Orchestrator process:

```
jcmd <pid> JFR.start duration=60s filename=orchestrator-debug.jfr
```

3. **Analyze:** Extract the `.jfr` file and load it into a tool like JDK Mission Control to visualize thread activity, garbage collection, and socket connections.

Hands-On Activity: Analyzing Workflow Logs

Follow these steps to isolate a failing workflow deployment.

1. **Step 1: Check Pod Logs** Identify the Orchestrator pod and inspect the recent output: `+ [source,bash] --- oc get pods -n rhdh-namespace oc logs <orchestrator-pod-name> -n rhdh-namespace --tail=100 ---`
2. **Step 2: Inspect Config Maps** Verify that your workflow definition has been injected correctly: `+`

```
[source,bash] ---- oc get configmap <workflow-config-name> -o yaml ----
```

- Step 3: Verify Web Service Connectivity** Ensure your workflow can communicate with external services. If your workflow relies on SOAP web services (JAX-WS), verify that the generated stubs are reachable and that the endpoint WSDL is accessible from within the cluster network.



If you suspect network policy interference, use `oc debug` to launch a temporary pod in the same namespace and attempt to `curl` the target web service endpoint to verify connectivity.

Next Steps

- If logs indicate serialization errors, verify your workflow input schema against the provided software template parameters.
- For persistent synchronization issues, ensure that your `Catalog` entities are correctly annotated so the Orchestrator can discover them.

Integrating Loki logs for workflow monitoring

Integrating Loki logs for workflow monitoring

Effective debugging is critical when managing complex serverless workflows in Red Hat Developer Hub (RHDH). By integrating Loki as a log provider, you gain centralized visibility into your Orchestrator workflow executions directly within the RHDH interface, allowing for faster root-cause analysis and performance monitoring.

Prerequisites

- An active Red Hat Developer Hub instance deployed on OpenShift.
- Access to an existing Loki instance within your OpenShift cluster.
- Cluster-admin or equivalent permissions to modify ConfigMaps.
- The `oc` CLI tool authenticated to your cluster.

Configuring Loki for Orchestrator

The integration process involves enabling the specific backend module and configuring the connection settings to point Orchestrator toward your Loki instance.



Ensure your Loki instance is reachable from the RHDH pod. If you are using self-signed certificates, ensure `rejectUnauthorized` is set to `false` during the configuration step.

Step 1: Enable the Loki Backend Module

You must first enable the Loki backend plugin within your dynamic plugins configuration.

1. Navigate to your OpenShift console and locate the `redhat-developer-hub-dynamic-plugins` ConfigMap.
2. Switch to the **YAML** view.
3. Add the Loki backend module to the `plugins` section:

```
- disabled: false
  package: oci://registry.access.redhat.com/rhdh/red-hat-developer-hub-backstage-
  pluginorchestrator-backend-module-loki:{{inherit}}
```

1. Save the changes. RHDH will automatically trigger a pod rollout to apply the new plugin.

Step 2: Configure the Log Provider

Next, you must configure the Orchestrator service to communicate with the Loki API.

1. Retrieve your Loki base URL by executing the following command in your terminal:

```
LOKI_HOST=$(oc get route logging-loki -n openshift-logging -o
jsonpath='{.spec.host}')
echo "https://$LOKI_HOST/api/logs/v1/application/"
```

2. Retrieve your authentication token by running:

```
oc whoami -t
```

3. Open your `redhat-developer-hub-app-config` ConfigMap and update the `orchestrator` section with the following configuration:

```
orchestrator:
  workflowLogProvider:
    loki:
      baseUrl: <LOKI_BASE_URL> # Replace with the output from step 1
      token: <AUTH_TOKEN>      # Replace with the output from step 2
      rejectUnauthorized: false
```

1. Save the ConfigMap. Once the pods restart, the Orchestrator backend will begin routing workflow logs through Loki.

Troubleshooting and Optimization

If logs do not appear in the interface, check the following:

- **Connectivity:** Verify that the RHDH pod can resolve the `LOKI_HOST` URL.
- **Permissions:** Ensure the `token` provided has sufficient RBAC permissions to read logs from the `openshift-logging` namespace.

- **Filtering:** If you have high log volume, you can further refine the data returned by uncommenting the `logPipelineFilters` or `logStreamSelectors` in the `app-config.yaml` file to target specific application labels or regex patterns.



For debugging, use the `logStreamSelectors` field to isolate logs to specific workflow IDs or namespace labels, significantly reducing the "noise" when troubleshooting intermittent workflow failures.

Appendix

FIXME: Content for Appendix...

Resources

[Download Course PDF](#)